

领域驱动、领域模型与 Actor 模型-概念讲解

1. 总览

领域驱动设计、领域模型和 Actor

模型经常一起出现在复杂业务系统、分布式系统和高并发系统中，但它们解决的问题不同。

一句话区分：

领域驱动设计：一种面向复杂业务的软件设计方法。

领域模型：对业务概念、规则和行为的抽象表达。

Actor 模型：一种面向并发和分布式的计算模型。

三者关系可以理解为：

DDD 帮你划清业务边界；

领域模型帮你表达业务规则；

Actor 模型帮你承载并发状态和消息交互。

2. 领域驱动设计 DDD

领域驱动设计，Domain-Driven Design，简称 DDD，是一种面向复杂业务系统的软件设计思想。

DDD 的核心目标是：

让软件模型尽量贴近业务领域本身，而不是只围绕数据库表或技术框架组织代码。

它强调开发人员与业务专家使用统一语言，共同建立领域模型，并围绕业务边界设计系统。

3. DDD 解决什么问题

DDD 主要解决复杂业务系统中的问题：

- 业务规则复杂，代码难以理解。
- 模块边界不清，改一个功能影响很多地方。
- 数据库表驱动开发，业务行为散落在各个 service 中。
- 技术模型和业务模型不一致，沟通成本高。
- 系统越来越大后，难以拆分、演进和维护。

DDD 不适合所有项目。对于简单 CRUD

系统，直接分层架构可能更高效；对于业务规则复杂、长期演进的系统，DDD 更有价值。

4. DDD 的两个层面

DDD 通常分为战略设计和战术设计。

4.1 战略设计

战略设计关注系统边界和业务划分。

常见概念：

- 领域
- 子域
- 限界上下文
- 上下文映射
- 通用语言

4.2 战术设计

战术设计关注具体代码模型。

常见概念：

- 实体
- 值对象
- 聚合
- 聚合根
- 仓储
- 领域服务
- 领域事件
- 应用服务

考试和架构讨论中，常考的是“DDD 用于复杂领域建模，强调限界上下文、统一语言和领域模型”。

5. DDD 核心概念

5.1 领域

领域是系统要解决的问题空间。

例如，在手机游戏服务端中，领域可能包括：

- 账号领域
- 角色领域
- 背包领域
- 商城领域
- 支付领域
- 活动领域
- 排行榜领域
- 匹配领域

5.2 子域

子域是领域的进一步拆分。

常见类型：

- 核心子域：系统竞争力所在
- 支撑子域：支撑核心业务
- 通用子域：通用能力，可复用或采购

游戏例子：

- 核心子域：战斗、匹配、活动玩法
- 支撑子域：背包、任务、排行榜
- 通用子域：账号、日志、邮件、配置

5.3 通用语言

通用语言是业务人员和开发人员共同使用的一套词汇。

例如：

角色、道具、背包格、活动周期、奖励结算、赛季、排行榜、订单、补单

代码中的类名、方法名、接口名应尽量使用这些业务词汇，减少“技术语言”和“业务语言”割裂。

5.4 限界上下文

限界上下文是一个明确的模型边界。

同一个词在不同上下文中可能含义不同。

例如“角色”：

战斗上下文：角色关注属性、技能、Buff、战斗状态。
社交上下文：角色关注昵称、头像、公会、好友关系。
支付上下文：角色关注充值资格、订单归属、风险控制。

DDD 不要求全系统只有一个统一大模型，而是允许不同上下文中存在不同模型。

6. 领域模型

领域模型是对业务领域中概念、规则、关系和行为的抽象。

它不只是数据结构，也不等于数据库表。

领域模型应该表达：

- 业务对象。
- 对象之间的关系。
- 业务规则。
- 状态变化。
- 关键行为。

例如，背包领域模型不应只包含 ItemId、Count 字段，还应表达：

添加道具
消耗道具
检查容量
检查叠加上限
绑定道具规则
过期道具清理

7. 领域模型与数据库模型的区别

对比项	领域模型	数据库模型
关注点	业务概念和业务行为	数据存储和查询
主要目标	表达业务规则	持久化和检索数据
典型形式	类、对象、聚合、领域服务	表、字段、索引、外键
设计依据	业务语义	存储效率和数据一致性

| 是否包含行为 | 应该包含 | 通常不包含 |

易错点：

领域模型不是表结构的简单映射。

如果一个对象只有 getter/setter，没有业务行为，通常只是贫血模型。

8. 领域模型的常见组成

8.1 实体

实体有唯一标识，生命周期中状态会变化。

例子：

玩家角色

订单

公会

活动实例

判断标准：

即使属性变化，只要 ID 不变，它仍然是同一个对象。

8.2 值对象

值对象没有独立身份，通常通过属性值判断相等。

例子：

坐标

价格

时间范围

属性加成

道具配置

值对象通常应设计为不可变对象。

8.3 聚合与聚合根

聚合是一组强一致性的领域对象集合。

聚合根是外部访问聚合的唯一入口。

例子：

背包聚合：Bag 是聚合根，BagItem 是聚合内部对象。

订单聚合：Order 是聚合根，OrderLine 是聚合内部对象。

公会聚合：Guild 是聚合根，Member 是聚合内部对象。

规则：

外部不要直接修改聚合内部对象，应通过聚合根执行业务方法。

8.4 仓储

仓储用于保存和获取聚合。

它隐藏数据库、缓存、文件等持久化细节。

例子：

```
BagRepository.Load(playerId)
BagRepository.Save(bag)
```

仓储不是业务规则的承载者，业务规则应尽量放在领域模型中。

8.5 领域服务

当一个业务行为不适合放在某个实体或值对象中时，可以使用领域服务。

例子：

```
跨玩家匹配服务
跨账户风控检查
复杂活动奖励计算
```

领域服务仍然属于领域层，应表达业务规则，而不是简单包装数据库操作。

8.6 领域事件

领域事件表示领域中已经发生的重要事实。

例子：

```
PlayerLeveledUp
OrderPaid
ItemConsumed
SeasonEnded
GuildCreated
```

领域事件常用于解耦后续动作：

```
订单支付成功 -> 发放道具
角色升级 -> 推送成就
赛季结束 -> 结算排行榜奖励
```

9. Actor 模型

Actor 模型是一种并发计算模型。

它把系统看成一组相互独立的 Actor，每个 Actor 拥有自己的状态，并通过消息进行通信。

Actor 的基本特征：

```
封装状态
通过消息交互
串行处理消息
避免共享内存
天然适合并发和分布式
```

一个 Actor 收到消息后可以：

```
改变自己的状态
发送消息给其他 Actor
创建新的 Actor
```

决定下一步处理行为

10. Actor 模型解决什么问题

Actor 模型主要解决并发和分布式状态管理问题。

适用场景：

- 大量独立对象并发运行。
- 每个对象有自己的状态。
- 对象之间通过异步消息交互。
- 希望减少锁和共享内存竞争。
- 系统需要横向扩展。

手机游戏服务端是典型场景：

每个玩家可以是一个 PlayerActor。

每个房间可以是一个 RoomActor。

每个公会可以是一个 GuildActor。

每个排行榜分片可以是一个 RankActor。

11. Actor 模型核心概念

11.1 Actor

Actor 是并发系统中的最小自治单元。

它通常包含：

身份 ID
内部状态
消息处理逻辑
邮箱
生命周期

11.2 消息

Actor 之间不直接共享状态，而是发送消息。

例子：

AddItem
PayOrder
JoinRoom
StartMatch
UpdateScore

11.3 邮箱

邮箱用于保存等待处理的消息。

通常一个 Actor 对自己的消息按顺序处理，因此可以减少显式锁的使用。

11.4 状态封装

Actor 的状态由 Actor 自己维护。

外部不能直接修改 Actor 状态，只能发送消息请求 Actor 自己修改。

这与 DDD 中“通过聚合根维护一致性”的思想很接近。

11.5 位置透明

在分布式 Actor 框架中，调用者通常不需要关心 Actor 在本机还是远程节点。

例如 Orleans 的虚拟 Actor 思想：

通过 Actor/Grain ID 调用对象；
运行时负责激活、定位、调度和回收。

12. DDD 与 Actor 模型的关系

DDD 和 Actor 模型不是同一类东西。

DDD 是业务建模方法。
Actor 是并发和分布式计算模型。

但它们可以很好地结合。

典型结合方式：

限界上下文 -> 服务边界
聚合根 -> Actor
领域事件 -> Actor 消息或事件流
领域行为 -> Actor 消息处理方法
仓储 -> Actor 状态持久化

例如：

PlayerActor 承载玩家聚合；
BagActor 承载背包聚合；
RoomActor 承载房间聚合；
GuildActor 承载公会聚合。

13. 游戏服务端例子

假设系统是一个移动游戏服务端，包含角色、背包、商城、活动和排行榜。

13.1 DDD 划分

可以划分为：

角色上下文
背包上下文
商城上下文
活动上下文
排行榜上下文

每个上下文有自己的模型和业务规则。

13.2 领域模型

背包领域模型可以包括：

Bag：背包聚合根
BagItem：背包物品
ItemConfig：道具配置值对象
AddItem：添加道具行为
ConsumeItem：消耗道具行为
BagFull：背包满事件
ItemConsumed：道具消耗事件

13.3 Actor 实现

可以用 Actor 承载聚合状态：

BagActor(playerId)

它接收消息：

AddItemCommand
ConsumeItemCommand
GetBagQuery

处理方式：

Actor 串行处理玩家背包消息；
领域模型校验业务规则；
状态变更后持久化到 MySQL 或缓存；
必要时发布领域事件。

这样可以减少同一玩家背包并发修改时的锁竞争。

14. 优点与风险

14.1 DDD 优点

- 业务边界清晰。
- 模型贴近业务。
- 复杂规则更容易维护。
- 有利于微服务拆分。
- 有利于长期演进。

14.2 DDD 风险

- 初期建模成本较高。
- 简单 CRUD 项目可能过度设计。
- 团队如果不了解业务，容易建出空洞模型。
- 容易把 DDD 误用成目录结构，而不是业务建模方法。

14.3 Actor 模型优点

- 适合高并发。
- 状态封装好。

- 减少共享内存和锁。
- 适合分布式扩展。
- 适合游戏、IoT、实时协作等场景。

14.4 Actor 模型风险

- 消息链路复杂后排查困难。
- 分布式调用仍然可能失败。
- Actor 粒度过细会增加调度成本。
- 跨 Actor 事务要谨慎设计。
- 不能把所有对象都机械地设计成 Actor。

15. 三者对比

名称	类型	关注点	典型问题
DDD	领域驱动设计	设计方法	业务边界和复杂业务建模 系统怎么按业务拆分
领域模型	业务抽象	业务对象、规则和行	业务规则放在哪里
Actor 模型	并发模型	状态封装、消息通信、分布式并发	高并发状态怎么安全处理

16. 考试答题模板

如果题目问 DDD、领域模型或 Actor 模型，可以按下面模板回答：

领域驱动设计是一种面向复杂业务的软件设计方法，强调通过通用语言、限界上下文和领域模型来表达业务知识。领域模型是对业务概念、业务规则和业务行为的抽象，通常由实体、值对象、聚合、领域服务和领域事件等组成。Actor 模型是一种并发计算模型，每个 Actor 封装自身状态，通过异步消息通信，避免共享内存带来的并发冲突。在复杂分布式系统中，可以用 DDD 划分业务边界，用领域模型表达业务规则，用 Actor 模型承载高并发状态对象，从而提高系统的可扩展性。

17. 最终总结

DDD 解决“业务怎么建模”。
领域模型解决“业务规则怎么表达”。
Actor 模型解决“并发状态怎么运行”。

三者结合时，一个常见思路是：

用 DDD 找边界；
用领域模型写规则；
用 Actor 承载有状态、高并发、可独立演进的聚合对象。